

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence and Application Development

MODULE - 1

Practical – I

Roll No.: T032	Name: KHUSHI
Class: TYIT	Batch: 2
Date of Assignment: 27-06-26	Date/Time of Submission: 01-07-26

AIM:- Design a simple Rational Agent by defining PEAS for Vacuum Cleaner Environment & Autonomous Taxi. Classify environment (fully/partially observable, deterministic/stochastic , episodic/sequential).Implement a simple table-driven agent in python.

A] Vacuum Cleaner Environment :-

CODE:-

```
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Move ',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Move',
    ('C', 'Dirty'): 'Suck',
    ('C', 'Clean'): 'Move',
    ('D', 'Dirty'): 'Suck',
    ('D', 'Clean'): 'Move'
}
sequence_table = {
    (('A', 'Clean'),('B', 'Clean'),('C', 'Clean'),('D', 'Clean')): 'NoOp',
    (('A', 'Dirty'),('B', 'Clean'),('C', 'Clean'),('D', 'Clean')): 'Suck A',
    (('A', 'Clean'),('B', 'Dirty'),('C', 'Clean'),('D', 'Clean')): 'Suck B',
    (('A', 'Clean'),('B', 'Clean'),('C', 'Dirty'),('D', 'Clean')): 'Suck C',
    (('A', 'Clean'),('B', 'Clean'),('C', 'Clean'),('D', 'Dirty')): 'Suck D'
}
percepts = []
def table_driven_agent(percept):
    percepts.append(percept)
    # Check if the complete percept sequence matches
    if tuple(percepts) in sequence_table:
        return sequence_table[tuple(percepts)]
    return table.get(percept, "NoOp")
print("4-Room Vacuum Cleaner Table-Driven Agent")
n = int(input("Enter number of percepts (4): "))
for i in range(n):
    print(f"\nPercept {i+1}")
    location = input("Enter Location (A/B/C/D): ").upper()
    status = input("Enter Status (Clean/Dirty): ").capitalize()
    percept = (location, status)
    action = table_driven_agent(percept)
```

```
print("Percept History :", percepts)
print("Agent Action :", action)
```

OUTPUT:-

```
4-Room Vacuum Cleaner Table-Driven Agent

Percept 1
Percept History : [('A', 'Clean')]
Agent Action    : Move

Percept 2
Percept History : [('A', 'Clean'), ('B', 'Dirty')]
Agent Action    : Suck

Percept 3
Percept History : [('A', 'Clean'), ('B', 'Dirty'), ('C', 'Clean')]
Agent Action    : Move

Percept 4
Percept History : [('A', 'Clean'), ('B', 'Dirty'), ('C', 'Clean'), ('D ', 'Clean')]
Agent Action    : NoOp
```

B] Autonomous Taxi :-

CODE:-

```
table = {
('Green', 'No', 'No', 'No', 'No'): 'Move Forward',
('Green', 'Yes', 'No', 'No', 'No'): 'Stop',
('Red', 'No', 'No', 'No', 'No'): 'Stop',
('Yellow', 'No', 'No', 'No', 'No'): 'Slow Down',
('Green', 'No', 'Yes', 'No', 'No'): 'Pick Up Passenger',
('Red', 'Yes', 'No', 'No', 'No'): 'Stop',
('Yellow', 'Yes', 'No', 'No', 'No'): 'Stop',
('Green', 'No', 'No', 'Yes', 'No'): 'Give Way to Emergency Vehicle',
('Green', 'No', 'No', 'No', 'Yes'): 'Go to Fuel Station'
}

percepts = []
def taxi_agent(percept):
    percepts.append(percept)
    action = table.get(percept, "Wait")
    return action
print("Autonomous Taxi Table-Driven Agent")
n = int(input("Enter number of percepts: "))
for i in range(n):
    print(f"\nPercept {i+1}")
    signal = input("Traffic Signal (Green/Yellow/Red): ").capitalize()
    obstacle = input("Obstacle Present? (Yes/No): ").capitalize()
    passenger = input("Passenger Request? (Yes/No): ").capitalize()
    emergency = input("Emergency Vehicle Nearby? (Yes/No): ").capitalize()
    fuel = input("Low Fuel? (Yes/No): ").capitalize()
    percept = (signal, obstacle, passenger, emergency, fuel)
    action = taxi_agent(percept)
    print("Current Percept:", percept)
    print("Agent Action:", action)
    print("khushii T032")
```

OUTPUT:-

```

Autonomous Taxi Table-Driven Agent

Percept 1
Current Percept: ('Red', 'No', 'No', 'No', 'No')
Agent Action: Stop
khushi T032

Percept 2
Current Percept: ('Green', 'No', 'Yes', 'No', 'No')
Agent Action: Pick Up Passenger
khushi T032

Percept 3
Current Percept: ('Green ', 'No', 'No', 'Yes', 'No')
Agent Action: Wait
khushi T032

Percept 4
Current Percept: ('Yellow', 'Yes', 'No', 'No', 'No')
Agent Action: Stop
khushi T032

Percept 5
Current Percept: ('Green ', 'No', 'No', 'Yes', 'Yes')
Agent Action: Wait
khushi T032

```

C] Here's a simple Table-Driven Agent program for a Smart Fan/Light System. It turns the fan/light ON or OFF depending on:

- **Whether someone is in the room.**
- **Whether the person has left the room.**
- **Whether natural light or natural air is available**

CODE:-

```

table = {
    # (Person in Room, Left Room, Natural Light/Air)
    ('Yes', 'No', 'No'): 'Turn ON Fan/Light',
    ('Yes', 'No', 'Yes'): 'Turn OFF Fan/Light',
    ('No', 'Yes', 'No'): 'Turn OFF Fan/Light',
    ('No', 'Yes', 'Yes'): 'Turn OFF Fan/Light',
    ('No', 'No', 'No'): 'Keep OFF',
    ('No', 'No', 'Yes'): 'Keep OFF',
    ('Yes', 'Yes', 'No'): 'Turn OFF Fan/Light',
    ('Yes', 'Yes', 'Yes'): 'Turn OFF Fan/Light'
}

percepts = []
def smart_home_agent(percept):
    percepts.append(percept)
    action = table.get(percept, "No Action")
    return action
print("Smart Fan/Light Table-Driven Agent")
n = int(input("Enter number of percepts: "))
for i in range(n):
    print(f"\nPercept {i+1}")
    person = input("Is a person in the room? (Yes/No): ").capitalize()
    left_room = input("Has the person left the room? (Yes/No): ").capitalize()
    natural = input("Is natural light/air available? (Yes/No): ").capitalize()
    percept = (person, left_room, natural)
    action = smart_home_agent(percept)

```

```
print("Current Percept:", percept)
print("Agent Action:", action)
print("khushii T032")
```

OUTPUT:

```
Smart Fan/Light Table-Driven Agent

Percept 1
Current Percept: ('Yes', 'No', 'No')
Agent Action: Turn ON Fan/Light

Percept 2
Current Percept: ('No', 'No', 'Yes')
Agent Action: Keep OFF

Percept 3
Current Percept: ('Yes', 'Yes', 'Yes')
Agent Action: Turn OFF Fan/Light
khushii T032
```